

Соответствие Карри-Ховарда

Гейне М.А. (geine@bmstu.ru)

05.12.2025

Предисловие

- С вычислительной техникой неразрывно связана тема искусственного интеллекта
- Одним из важных компонентов ИИ является "логика"
- Логика в вычислительной и математической среде представляется в виде записанных теорем, их доказательства и проверки правильности этих доказательств
- В компьютерных науках для изучения логических теорем строятся программы - анализаторы теорем
- В период активного исследования анализаторов теорем было выявлено некоторое интересное соответствие, о котором пойдёт речь

Акт I. Типы и высказывания

Три простые функции

- `let apply f x = f x`
- `let const x = fun _ -> x`
- `let subst x y z = x z (y z)`

Три простые функции

- `let apply f x = f x`
`: ( 'a -> 'b) -> 'a -> 'b`
- `let const x = fun _ -> x`
`: 'a -> 'b -> 'a`
- `let subst x y z = x z (y z)`
`: ('a -> 'b -> 'c) -> ('a -> 'b) -> 'a -> 'c`

Три простые функции

- `let apply f x = f x`
 `: ( 'a -> 'b) -> 'a -> 'b`
- `let const x = fun _ -> x`
 `: 'a -> 'b -> 'a`
- `let subst x y z = x z (y z)`
 `: ('a -> 'b -> 'c) -> ('a -> 'b) -> 'a -> 'c`

Три простые функции высказывания

- `let apply f x = f x`
: `('a ⇒ 'b) ⇒ 'a ⇒ 'b`
- `let const x = fun _ -> x`
: `'a ⇒ 'b ⇒ 'a`
- `let subst x y z = x z (y z)`
: `('a ⇒ 'b ⇒ 'c) ⇒ ('a ⇒ 'b) ⇒ 'a ⇒ 'c`

Три простые функции высказывания

- `let apply f x = f x`
: $(A \Rightarrow B) \Rightarrow A \Rightarrow B$
- `let const x = fun _ -> x`
: $A \Rightarrow (B \Rightarrow A)$
- `let subst x y z = x z (y z)`
: $(A \Rightarrow (B \Rightarrow C)) \Rightarrow ((A \Rightarrow B) \Rightarrow (A \Rightarrow C))$

Цели. Выведение методов формальной логики.

Формальная логика L.

1. $\mathcal{A} : \alpha A, B, C, \dots, A_1, \dots, C_{15}, \dots$

а) лог. связки $\neg, \rightarrow$

б) скобки

2. $\varphi : \alpha) \mathcal{A}. \alpha$

б) A, B — формулы, но $\neg A, A \rightarrow B$ — формулы

3. $\beta : \mathcal{A}_1 : A \rightarrow (B \rightarrow A)$

$\mathcal{A}_2 : (A \rightarrow (B \rightarrow C)) \rightarrow ((A \rightarrow B) \rightarrow (A \rightarrow C))$

$\mathcal{A}_3 : (\neg B \rightarrow \neg A) \rightarrow ((\neg B \rightarrow A) \rightarrow B)$

4. R — это МР (Modus Ponens), т.е. правило отсечения
ponendo

$$\frac{A, A \rightarrow B}{B} \text{ МР}$$

B отсечается по правилу МР.

Три простые функции/высказывания

- `let apply f x = f x`
: $(A \Rightarrow B) \Rightarrow A \Rightarrow B$ — Modus Ponens
- `let const x = fun _ -> x`
: $A \Rightarrow (B \Rightarrow A)$ — A1
- `let subst x y z = x z (y z)`
: $(A \Rightarrow (B \Rightarrow C)) \Rightarrow ((A \Rightarrow B) \Rightarrow (A \Rightarrow C))$ — A2

Типы и высказывания

Логические высказывания могут читаться так же, как типы в программах, и наоборот

Тип	Высказывание
Переменная типа 'a	Простое высказывание A
Тип функции ->	Импликация $\Rightarrow$

Конъюнкция и истина

- `let fst (a, b) = a`
 `: 'a * 'b -> 'a`
- `let snd (a, b) = b`
 `: 'a * 'b -> 'b`
- `let pair a b = (a, b)`
 `: 'a -> 'b -> ('a * 'b)`
- `let tt = ()`
 `: unit`

Конъюнкция и истина

- `let fst (a, b) = a`
 `: A ∧ B ⇒ A`
- `let snd (a, b) = b`
 `: A ∧ B ⇒ B`
- `let pair a b = (a, b)`
 `: A ⇒ (B ⇒ (A ∧ B))`
- `let tt = ()`
 `: true`

Дизъюнкция (ИЛИ)

Логическое **ИЛИ** в программировании — это выбор.

В OCaml это **Sum Type**:

- `type ('a, 'b) disj = Left of 'a | Right of 'b`
`: A ∨ B`

Это **конструктивное ИЛИ**:

Чтобы создать этот тип, вы обязаны предоставить либо доказательство **A** (`Left A`), либо доказательство **B** (`Right B`).

Ложь (False) и Пустота (Empty)

Ложь — это то, чего не может существовать.

- `type empty = |` (У типа нет конструкторов!)
 `: ⊥ (False)`
- Если вы напишете функцию, возвращающую `empty` :
- `let e : empty = failwith ""`
 `: Exception`

Ненаселенный тип = Недоказуемое утверждение = Ложь.

Отрицание (Negation)

В логике: $\neg A \iff A \Rightarrow \text{False}$

В программировании это функция, которая превращает **A** в **Empty**:

- `type 'a negation = 'a -> empty`

Как это работает?

Если у вас есть такая функция, это обещание: *"Дай мне значение типа A, и я дам тебе значение типа Empty"*.

Но так как **Empty** получить невозможно, это означает, что вы **никогда не сможете дать мне значение A.**

Следовательно, **A не существует.**

Розеттский камень: Python, OCaml и Логика

Чтобы понять магию, давайте переведем всё на язык, который вы знаете

Концепция	Python (Type Hints)	OCaml (ML)	Логика
Импликация	<code>Callable[[A], B]</code>	<code>'a -> 'b</code>	$A \Rightarrow B$
Конъюнкция (И)	<code>Tuple[A, B]</code>	<code>'a * 'b</code>	$A \wedge B$
Дизъюнкция (ИЛИ)	<code>Union[A, B]</code>	<code>'a 'b</code>	$A \vee B$
Истина	<code>None / ()</code>	<code>() (unit)</code>	$\top$ (True)
Ложь	<code>NoReturn / Never</code>	<code>empty</code>	$\perp$ (False)

“ В Python 3.11 тип `Never` (или `NoReturn`) означает функцию, которая **никогда** не вернет управление (например, вечный цикл или `raise Exception`). ”

Типы и высказывания

Логические высказывания могут читаться так же, как типы в программах, и наоборот

Тип	Высказывание
Переменная типа 'a	Простое высказывание A
Тип функции ->	Импликация $\Rightarrow$
Тип кортежа *	Конъюнкция $\wedge$
unit	True

**Типы программ и логические высказывания
фундаментально являются одним и тем же**

Акт II. Программы и доказательства

Правила типизации

- Мы имеем некоторое статическое окружение, которое отображает идентификаторы в типы
- Отношение вида $env \vdash e : t$ означает, что e имеет тип t в окружении env
- Для применения функции:
 $if\ env \vdash e1 : t \rightarrow u$
 $and\ env \vdash e2 : t$
 $then\ env \vdash e1\ e2 : u$

Правила типизации

- if $env \vdash e1 : t \rightarrow u$
- and $env \vdash e2 : t$
- then $env \vdash e1\ e2 : u$

Правила типизации

- if $env \vdash e1 : t \rightarrow u$
- and $env \vdash e2 : t$
- then $env \vdash e1\ e2 : u$

Правила типизации

- if $env \vdash e1 : t \Rightarrow u$
- and $env \vdash e2 : t$
- then $env \vdash e1 \ e2 : u$

Правила типизации

- if $env \vdash e1 : t \Rightarrow u$
- and $env \vdash e2 : t$
- then $env \vdash e1\ e2 : u$

Modus Ponens:

$$A \Rightarrow B; A$$

— — — — —

$$B$$

Правила применения импликации и конъюнкции

$$\frac{\Gamma, A \vdash B}{\Gamma \vdash A \Rightarrow B} \Rightarrow \text{intro}$$

$$\frac{\Gamma \vdash A \Rightarrow B, \Gamma \vdash A}{\Gamma \vdash B} \Rightarrow \text{elim}$$

$$\frac{\Gamma \vdash A, \Gamma \vdash B}{\Gamma \vdash A \wedge B} \wedge \text{intro}$$

$$\frac{\Gamma \vdash A \wedge B}{\Gamma \vdash A} \wedge \text{elim 1}$$

$$\frac{\Gamma \vdash A \wedge B}{\Gamma \vdash B} \wedge \text{elim 2}$$

Правила типизации

if $env \vdash e1 : t \rightarrow u$
and $env \vdash e2 : t$
then $env \vdash e1 e2 : u$

Или

$$\frac{env \vdash e1 : t \rightarrow u \quad env \vdash e2 : t}{env \vdash e1 e2 : u}$$

Правила типизации

if $\text{env} \vdash e1 : t \Rightarrow u$
and $\text{env} \vdash e2 : t$
then $\text{env} \vdash e1\ e2 : u$

Или

$\text{env} \vdash e1 : t \Rightarrow u$
 $\text{env} \vdash e2 : t$
— — — — $\Rightarrow \text{elim}$
 $\text{env} \vdash e1\ e2 : u$

Вычисления с подтверждениями

- Если у нас есть функция, принимающая тип `int` на вход, и мы вызываем её с значением `42`, мы таким образом предоставляем функции подтверждение того, что у типа `int` есть значение
- Modus Ponens - способ вычисления с подтверждениями
 - Если у нас есть подтверждение того, что `e2` - это `t`
 - И если у нас есть способ `e1` преобразовать подтверждение `t` в подтверждение `u`
 - MP производит подтверждение `u`, применяя `e1` к `e2`, тем самым доказывая истинность `u`
- Таким образом, `e1 e2` является программой... и доказательством!

Правила типизации для пар

$$\text{env} \vdash e : t1 * t2$$

— — — —

$$\text{env} \vdash \text{fst } e : t1$$

Или:

$$\text{env} \vdash e : t1 * t2$$

— — — —

$$\text{env} \vdash \text{snd } e : t2$$

Правила доказательств для $\wedge$

$\text{env} \vdash e : t1 \wedge t2$

$\text{— — — — } \wedge \text{ elim } 1$

$\text{env} \vdash \text{fst } e : t1$

Или:

$\text{env} \vdash e : t1 \wedge t2$

$\text{— — — — } \wedge \text{ elim } 2$

$\text{env} \vdash \text{snd } e : t2$

Программы и доказательства

- Хорошо типизированная программа (т.е. с корректно применёнными типами) демонстрирует, что есть хотя бы одно значение для заданного типа
 - иными словами, этот тип *населён*
 - программа является *доказательством* того, что тип *населён*
- И это доказательство демонстрирует, что есть как минимум один способ вывода формулы
 - иными словами, формула доказуема с использованием предположений и правил вывода
 - доказательством является программа, которая использует подтверждения предположений
- **Программы являются доказательствами, и доказательства являются программами**

**Программы и логические доказательства
фундаментально являются одним и тем же**

Акт III. Оценка и упрощение

Много доказательств/программ

Любое высказывание/тип могут иметь множество доказательств/программ

Предположим, мы имеем программу: `fst (a, b)`

Разумеется, программа вернёт `a` . Рассмотрим вывод типов в программе

$\frac{}{\{a : t, b : t'\} \vdash a : t}$	$\frac{}{\{a : t, b : t'\} \vdash b : t'}$
$\frac{}{\{a : t, b : t'\} \vdash (a, b) : t * t'}$	
$\frac{}{\{a : t, b : t'\} \vdash \text{fst } (a, b) : t}$	

$\frac{}{t, t' \vdash t}$	$\frac{}{t, t' \vdash t'}$
$\frac{}{t, t' \vdash t \wedge t'}$	
$\frac{}{t, t' \vdash t}$	

Но можно найти и более простое доказательство:

$$\frac{}{t, t' \vdash t}$$

Иными словами, нам не нужно проходить через $t \wedge t'$ чтобы доказать t , если t уже имеется в наборе посылок

$$\frac{}{\{a : t, b : t'\} \vdash a : t}$$

Такой вывод справедлив для программы a , и именно к ней приводит оценка большей программы $\text{fst } (a, b)$

Много доказательств/программ

Другой пример. Пусть есть следующее высказывание/тип:

- $A \Rightarrow (B \Rightarrow (A \wedge B))$
- `'a -> ('b -> ('a * 'b))`

Доказательства/программы:

- `fun x -> fun y -> (fun z -> (snd z, fst z)) (y, x)`
- `fun x -> fun y -> (snd (y, x), fst (y, x))`
- `fun x -> fun y -> (x, y)`
- У всех программ здесь один и тот же тип, но разное решение. И вывод типов становится проще с уменьшением решения

**Оценка программы и упрощение
доказательства фундаментально являются
одним и тем же**

Заключение

Это всё одно и то же

Программирование	Логика
Типы	Высказывания
Программы	Доказательства
Оценка	Упрощение

Вычисление - есть логическое рассуждение

Спасибо за внимание

$(n \rightarrow \mathbb{N}) \rightarrow \star$ $\lambda x. x$ — *PROOF! Теперь ты функция! И ты функция! ВЫ ВСЕ ФУНКЦИИ!*

☹(ò_ó˘)☹ *Type Checker is my gym bro now.*

🛑🖐 (˘_˘) ... $\perp$ *Halt! You cannot construct False here.*

$(\odot_ \odot)\odot_ _ _ \odot$

"Wait... proofs are *what* now?!"

$(\smile \circ \square \circ) \smile \frown \lambda$

"My brain is now a type error."

ERROR 404: Brain not found. Reason: Type Mismatch: Expected 'Simple Python', found 'Fundamental Truth of the Universe' 🤯

